

오픈소스 기반 반도체 칩 종합 설계 2주차

[우주 침략자 게임 만들기]

권효재

이번 학습노트를 통해 진행된 우주 침략자 게임 만들기 바이브 코딩의 흐름을 파악하며 과제를 진행하고, 또 내가 직접 바이브코딩을 해본 경험을 통해 느낀 바를 정리해보았다.

과제 1. diff 명령을 이용한 코드 비교(TB, TLM 2개의 파일 개선 전/후)

이번 과제에서는 diff 명령을 이용해서 제미나이의 수정 전, 후 코드를 비교해 볼 수 있었다. 확인해보니 sc_invaders_TB.cpp 파일은 두 코드 모두 키보드 입력을 받아 btn_left, btn_right, btn_fire 신호를 전달하는 기능은 같았고, 수정된 코드에서는 여러 개의 if-else문을 삼항 연산자로 줄여서 더 간단하게 정리되어 있었다. sc_glcd128x64_TLM.cpp 파일도 CLCD 출력하는 기본 동작은 같았다. 반복적인 부분이 줄고 코드가 전체적으로 정리된 것을 확인할 수 있었다.

이번 비교를 통해 기능을 제미나이나 다른 AI에게 수정을 맡겼을 때 코드가 어떻게 바뀌는지 직접 확인해볼 수 있었다. 특히 테스트벤치 파일에서는 실제 키보드 입력을 받아 btn_left, btn_right, btn_fire 신호로 전달하는 구조가 정리되어 있었고, 같은 기능을 유지하면서 코드가 더 간단하게 바뀌는 것도 확인할 수 있었다. 다만 수정된 코드가 실제로 같은 기능을 유지하는지는 동작해서 검증해보는 과정이 필요한데, AI가 단순히 코드를 최적화 해주는 것뿐만 아니라 테스트벤치의 입력 처리 구조나 코드 가독성을 좋게 해주기 때문에 이 개선된 가독성을 통해서 코드를 한번 더 사람이 검증하는 과정이 필요하다고 느꼈다.

과제 2. Makefile 오류 수정 및 빌드

처음 Makefile.mk를 실행했을 때 space_invaders_engine_glcd.v 파일을 찾지 못하는 오류가 발생했다. 확인해보니 Makefile에 적힌 파일 경로가 실제 위치와 달라서 경로를 수정했다. 이후에는 Verilator가 생성한 파일 이름과 Makefile에 적힌 이름이 달라서 다시 오류가 발생했고, 실제 top module 이름인 space_invaders_engine_glcd에 맞게 수정했다. 또 sc_main.cpp와 여러 헤더 파일이 현재 폴더에 없어서 이전 단계 폴더에서 필요한 파일을 복사했다. 그 다음에는 Verilog의 여러 비트 신호와 SystemC의 sc_uint 타입이 맞지 않는 오류가 발생했다. 이를 해결하기 위해 Verilator 옵션에 --pins-sc-uint를 추가했고, 최종적으로 빌드에 성공했다. 마지막에는 libsystemc.so.3.0 라이브러리를 찾지 못하는 문제가 있었지만 LD_LIBRARY_PATH를 설정한 뒤 정상적으로 실행되었고, SDL2 창에서 게임 화면이 출력되는 것을 확인했다.

이번 과정을 통해서 코드뿐만 아니라 파일 경로, 모듈 이름, 자료형, 라이브러리 설정 같은 환경도 중요하다는 것을 확인할 수 있었다. 오류가 여러 번 발생했지만 에러 메시지를 하나씩 확인하면서 원인을 수정해 나가는 과정에서, SystemC와 Verilog를 연결할 때 어떤 부분을 맞춰줘야 하는지 이해할 수 있었다. 특히 빌드 성공과 실행 성공은 별개의 문제라는 점도 알 수 있었다.

과제 3. 인공지능이 반도체 설계에 주는 도움

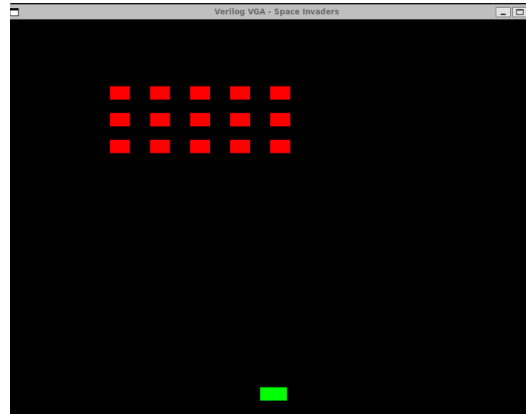
학습자료와 별개로, 혼자 바이브 코딩을 진행해 보았는데, 처음부터 상세하게 지시를 했지만 어떤 방법을 이용해야하는지, 이런 디테일들이 많이 부족했다.

Verilog RTL로 간단한 Space Invaders 게임을 만들어줘.
조건은 다음과 같아.

- FPGA에서 합성 가능한 RTL 코드로 작성
- 동기식 디지털 회로 구조 사용
- 플레이어는 좌우로 이동 가능
- 버튼을 누르면 총알을 발사
- 외계인이 화면에 표시됨
- VGA 640x480 화면을 기준으로 함
- 현재 VGA pixel 좌표인 vga_x, vga_y와 video_on을 입력으로 받음
- RGB 출력을 생성
- 게임 상태 제어는 FSM을 사용

우선 복잡한 그래픽이나 사운드는 제외하고 기본 게임 동작부터 구현해줘.

코드 작성 후 각 module과 주요 register, FSM의 역할도 설명해줘.



나의 바이브 코딩프롬프트(gpt)

중간 결과물

gpt에게 verilog로 게임을 만들어달라고 지시를 하니, 당연하게도 ai는 테스트 벤치도 verilog 자체로 구성을 해서 결과물을 내 놓았고, 따로 지시하지 않아 픽셀 전달도 systemC실행 환경이 아닌 VPI 호출 방식으로 진행했다. C++ 프로그램이 계속 시뮬레이터를 감시하는 게 아니라, 시뮬레이터가 이벤트가 생길 때 C++ 쪽에 알려주는 방식으로 진행한 것이다. 호출 횟수 때문에 그런지 결과물의 속도차가 많이 났다. VPI 방식보다 나은 DPI-C 방식을 학습노트에서 제미나이가 교수님께 추천을 했고, 그에 따라서 진행을 했지만 이후 DPI-C와 synstemC를 비교해달라고 했을 때 제미니가 본인이 이전에 추천했던 방식이 아닌, 초기의 간단한 C++ 연동에는 적합했지만, 주변 장치와 테스트벤치까지 포함한 SOC 환경에서는 syntemC가 더 낫다는 답변을 한것이 인상깊었다. 비교해달라고 부탁한 후에 더 나은 선택지를 알 수 있는 부분이, AI가 처음부터 여러 선택지 중 최적의 방법을 생각하는 것 보다 연산을 덜 하기 위해서 진행한다는 느낌이 들었다. OpenCV도 제미니의 추천에 의해서 그래픽 출력 방법으로 올라갔었지만, 결국에는 최적으로 SDL2 방식을 사용했던 것도 마찬가지이다. 이부분은 AI 관련한 수업을 듣거나 자료를 찾아보고 학습과 추론 방법을 더 공부해보아야 더 잘 이해할 수 있겠다는 생각이 들었다.

바이브 코딩에서 중요하게 처음부터 전체적인 구조와 이 구조를 구성하기 위해서 이용하는 여러 가지 방법들을 처음부터 상세히 짜 놓지 않으면 그 과정에서 수많은 수정이 필요하다는 것을 느꼈다. 하지만 항상 최적의 방법을 알고 시작하는 것이 아니라면, 설계하면서 구체화하는 과정에서 많은 설계 방법과 수정이 필요할 수 있고 그 과정에서 토큰 소모가 많아질 수 있다고 느꼈다. 학습노트의 바이브코드 과정 중간에 제미니가 맞춤형으로 교수님의 깃 ETRI DesignKit과 GitHub 프로젝트를 보고서 테스트벤치를 c++, systemC를 이용하는 방식을 추천했던 것처럼, 이미 프롬프트를 입력시키기 전에 기존에 진행했던 설계 환경이나 추구하는 방법들을 학습시킨 상태에서 원하는 방향으로 바이브 코딩하면 수정이나 검증이 덜 필요하지만, 그렇지 않다면 AI 모델에 따라 본인이 생각하는 방법, 방향을 지시를 잘 해야한다는 생각이 들었다. 전적으로 맡기는 것은 힘들지만 내가 잘 설계할 수 있는 실력이 되고 뭐가문제인지 볼수있는 안목이 생긴다면 좋은 도구가 될 수 있을 것 같다.